Centro Universitário de Viçosa – UNIVIÇOSA
Disciplina: INF101- Programação de Computadores I
Professores: Anderson R. Lamas e Vanderlea Queiroz

Trabalho – Implementação de um Sistema de Registro e Gestão de Contas Bancárias (ETAPA 2)

1. Pontuação e Entrega do Trabalho

Conteúdo: Etapa 2, conforme descrito neste documento.
Valor: 8 pontos
Data: 11/11/2026
Formato de entrega:

- Organizar seu trabalho no Github e compartilhar com o professor na planilha: https://docs.google.com/spreadsheets/d/14K47GMy3npZbMZ1fR_jms-tSPhS1aONOs4wQDQr395k/edit?gid=0#gid=0
- Mostrar o programa funcionando no laboratório na data descrita acima, ou se preferer no dia 04/11/26.

2. Etapa 2 – Descrição

Nesta etapa, o sistema deverá ser ampliado para permitir várias contas e registrar suas movimentações em memória. O aluno deverá perceber que criar numeroConta1, numeroConta2, numeroConta3 e assim por diante não é uma solução escalável. Os arrays serão utilizados para representar conjuntos de contas e movimentações. Não é obrigatório criar funções próprias: as operações podem ser implementadas diretamente no main(), utilizando estruturas de decisão e repetição.

2.1 Dados das contas

```
const int MAX_CONTAS = 100;  // é previsto um limite máximo de 100 contas

int numeroConta[MAX_CONTAS];
string nomeCliente[MAX_CONTAS];
string cpf[MAX_CONTAS];
int tipoConta[MAX_CONTAS];
double saldo[MAX_CONTAS];
bool contaAtiva[MAX_CONTAS];

int quantidadeContas = 0;
```

2.2 Histórico de movimentações

O sistema deverá manter as movimentações em memória. Cada posição dos arrays representa uma movimentação.

```
const int MAX_MOVIMENTACOES = 500;

int contaMovimentacao[MAX_MOVIMENTACOES];
char tipoMovimentacao[MAX_MOVIMENTACOES];
double valorMovimentacao[MAX_MOVIMENTACOES];

int quantidadeMovimentacoes = 0;
```

Código	Movimentação	Efeito no saldo
D	Depósito	+ valor
S	Saque	- valor
T	Transferência	Pode ser registrada como saída e entrada

2.3 Funcionalidades obrigatórias

1. Cadastrar contas.
2. Listar as contas cadastradas.
3. Consultar uma conta pelo número.
4. Realizar depósitos.
5. Realizar saques, verificando saldo suficiente.
6. Realizar transferências entre duas contas.
7. Exibir o extrato de uma conta.

OBS: As opções 4, 5 e 6 devem registrar as movientações

2.4 Exemplo de depósito

```
saldo[indice] = saldo[indice] + valor;

contaMovimentacao[quantidadeMovimentacoes] = numeroConta[indice];
tipoMovimentacao[quantidadeMovimentacoes] = 'D';
valorMovimentacao[quantidadeMovimentacoes] = valor;

quantidadeMovimentacoes++;
```

A operação altera o saldo da conta e registra uma nova movimentação no histórico. A busca da conta deverá ser realizada pelo número informado pelo usuário.

2.5 Exemplo de transferência entre duas contas

Considere duas contas cadastradas no sistema:

Conta 1001 – Ana: saldo inicial de R$ 1.000,00

Conta 1002 – Bruno: saldo inicial de R$ 500,00

Ana transfere R$ 300,00 para Bruno. Antes de efetuar a operação, o programa deve localizar as duas contas, verificar se ambas existem e estão ativas, confirmar que o valor é positivo e verificar se a conta de origem possui saldo suficiente.

Após a transferência, os saldos ficam assim:

Conta 1001 – Ana: R$ 1.000,00 − R$ 300,00 = R$ 700,00

Conta 1002 – Bruno: R$ 500,00 + R$ 300,00 = R$ 800,00

A transferência deve gerar dois registros no histórico de movimentações, um para cada conta:

Conta | Tipo | Valor

1001 | S | 300,00

1002 | D | 300,00

O registro S representa a saída de R$ 300,00 da conta de Ana; o registro D representa a entrada do mesmo valor na conta de Bruno. Embora sejam dois registros, eles correspondem a uma única transferência. A quantidade de movimentações deve ser incrementada em 2. Antes de gravar os registros, o programa deve garantir espaço para ambos no array de movimentações.

Exemplo conceitual da atualização dos saldos:

saldo[indiceOrigem] = saldo[indiceOrigem] - valor;

saldo[indiceDestino] = saldo[indiceDestino] + valor;

Os índices devem corresponder às posições encontradas para as contas 1001 e 1002 nos arrays de contas. O código acima ilustra apenas a atualização dos saldos; o aluno também deverá registrar as duas movimentações nos arrays do histórico.

2.6 Extrato

O extrato deverá percorrer o histórico e apresentar somente as movimentações pertencentes à conta consultada.

```
for (int i = 0; i < quantidadeMovimentacoes; i++) {
    if (contaMovimentacao[i] == numeroConta) {

        if (tipoMovimentacao[i] == 'D') {
            cout << "Deposito: R$ " << valorMovimentacao[i] << endl;
        }
        else if (tipoMovimentacao[i] == 'S') {
            cout << "Saque: R$ " << valorMovimentacao[i] << endl;
        }
    }
}
```

ATENÇÃO

Na Etapa 2, o histórico permanece apenas na memória. Ao fechar o programa, as contas e movimentações serão perdidas. A persistência será implementada na Etapa 3.

DESAFIO: Não é obrigatório modularizar o programa por meio de funções. O aluno pode implementar as operações diretamente no main(), organizando o código com comentários, indentação e blocos de decisão e repetição. Caso opte por organizer por funções, como seria implementadas:

```
void cadastrarConta();
void listarContas();
void consultarConta();
void depositar();
void sacar();
void transferir();
void registrarMovimentacao();
void exibirExtrato();
```